

Sumário

- Revisão do Plano de Melhoria Estrutural 4
 - Verificação de base 4
 - Erros factuais 5
 - 1. Afirmação falsa sobre `GEMINI.md` (linhas 18-20 do plano) 5
 - 2. Omissão de quatro arquivos de governança (linhas 18 e 33-37) .6
 - 3. Risco de regressão da documentação na Etapa 1 6
 - Propostas inviáveis ou de risco alto 6
 - 4. Symlinks de skills no Windows (linhas 51-64, 84-90) 6
 - 5. Centralização de `.mcp.json` sem considerar diferenças de schema (linhas 103-105) 7
 - Melhorias de clareza e precisão 7
 - 6. “Economia de mais de 80% de tokens de preâmbulo em cada interação” (linha 39) 7
 - 7. `sys.path` hack em `parametros_projeto.py` (linha 100) 8
 - 8. Plano não cita o `code-review-graph` na transição (Etapa 2) 8
 - 9. Aliases `@workspace` / `@skills` / `@brand` / `@output` (linhas 71-76) .8
 - 10. Gates de aceitação subjetivos (Etapa 1, linha 125) 8
 - Acertos que valem reiterar 9
 - Resumo executivo 10
 - Próximos passos sugeridos 12

Revisão do Plano de Melhoria Estrutural

Documento de revisão técnica do plano em `docs/04-plano-melhoria-estrutural.md`. O diagnóstico original está correto no essencial — há duplicidade real entre arquivos de governança e cópia física de skills entre plataformas —, mas o plano contém **erros factuais, omissões e propostas de risco** que comprometem a execução pretendida. As observações abaixo estão separadas por gravidade e todas foram verificadas contra o estado atual do repositório em 2026-08-05.

Verificação de base

Antes de emitir as observações, conferiu-se o estado real do repositório:

Arquivo / Diretório	Linhas / Conteúdo	Observação
CLAUDE.md	219 linhas	Cabeçalho + 9 regras + diagrama + tabela de módulos, ~95% idêntico a AGENTS.md
AGENTS.md	174 linhas	Mesmo núcleo que CLAUDE.md (fonte autoritativa carregada pelo harness)
GEMINI.md	46 linhas	Já enxuto, específico do Gemini — não compartilha ~150 linhas
CODEBUDDY.md	38 linhas	Omitido pelo plano
QODER.md	38 linhas	Omitido pelo plano
.cursorrules	38 linhas	Omitido pelo plano
.windsurfrules	38 linhas	Omitido pelo plano
.claude/skills/	16 skills (incl. 4 comuns)	Diretório físico
.gemini/skills/	4 skills comuns	Cópia física
.codebuddy/skills/	4 skills comuns	Cópia física
.mcp.json (raiz) + .cursor/mcp.json + .vscode/mcp.json + .kiro/settings/mcp.json + .qoder/mcp.json	5 arquivos	Schemas distintos por plataforma

Erros factuais

1. Afirmação falsa sobre GEMINI.md (linhas 18-20 do plano)

O diagnóstico afirma que “ CLAUDE.md , GEMINI.md , AGENTS.md ” compartilham “as mesmas ~150 linhas iniciais”. Hoje GEMINI.md tem **46 linhas** e já é específico do Gemini (Caveman Thinking, hooks de sessão, code-review-graph). A duplicidade massiva real é **apenas** entre CLAUDE.md (219) e AGENTS.md (174), que são ~95% idênticos.

Ação recomendada: corrigir as linhas 18-20 para refletir que `GEMINI.md` já está limpo e a duplicidade a atacar é `CLAUDE.md` ↔ `AGENTS.md`.

2. Omissão de quatro arquivos de governança (linhas 18 e 33-37)

O plano menciona apenas três arquivos de orquestração, mas o repo possui **seis**: também `CODEBUDDY.md`, `QODER.md`, `.cursorrules` e `.windsurfrules` (todos com 38 linhas, provavelmente espelhados entre si). A tabela de especialização (linha 33) está incompleta.

Ação recomendada: adicionar as quatro plataformas à tabela de especialização ou justificar explicitamente por que estão fora de escopo. Quatro arquivos de 38 linhas tipicamente idênticos representam o mesmo tipo de desperdício que o plano combate em `CLAUDE.md` / `AGENTS.md` — omiti-los enfraquece a proposta.

3. Risco de regressão da documentação na Etapa 1

A tabela de módulos hoje em `CLAUDE.md` / `AGENTS.md` contém a coluna “Selecionável via `/esbocar`” e o material “Textos de Apoio” (`redator-textos`, `validar-textos.py`, `output/<slug>/textos/`). A proposta de tornar `AGENTS.md` a “única fonte de verdade” precisa **preservar** esses acréscimos — caso contrário a refatoração de Etapa 1 regredirá a documentação em relação ao estado atual.

Ação recomendada: a Etapa 1 deve listar explicitamente o conteúdo a preservar (coluna “Selecionável via `/esbocar`” e linha “Textos de Apoio” da tabela de módulos) para evitar regressão.

Propostas inviáveis ou de risco alto

4. Symlinks de skills no Windows (linhas 51-64, 84-90)

`os.symlink` no Windows exige **Modo de Desenvolvedor ativo OU privilégios de Administrador** — o plano reconhece isso na linha 88, mas minimiza o impacto. Em máquinas de operador comuns e em runners CI do GitHub Actions no Windows, a criação falha silenciosamente, e skills somem do Claude Code/Gemini **sem aviso**.

Alternativa recomendada (ordem de preferência):

1. **Junction** (`mklink /J`) para diretórios — **não** exige admin nem Modo Desenvolvedor. É o mecanismo usado por `node_modules` do pnpm/npm para caches. Deveria ser o default, não `os.symlink`.
2. **Fallback à cópia com sync unidirecional** (`robocopy /rsync`) quando junction falhar — regenera a réplica a partir do master quando um checksum mudar.
3. O script de setup (linha 84) menciona “Limpa pastas físicas redundantes” (linha 90) — **perigoso**: se o link falhar e o script já tiver deletado a cópia física, a skill some do agente. O script deve ser **idempotente e abortar sem deletar** se a criação do link falhar.

5. Centralização de `.mcp.json` sem considerar diferenças de schema (linhas 103-105)

Existem cinco arquivos `.mcp.json` / `mcp.json` com **schemas distintos** por plataforma: `.mcp.json` (raiz), `.cursor/mcp.json`, `.vscode/mcp.json`, `.kiro/settings/mcp.json`, `.qoder/mcp.json`. Unificar em um template e copiar pode **quebrar** ferramentas cujo schema difere.

Ação recomendada: antes de propor unificação, documentar os schemas de cada plataforma ou aceitar que só dá para unificar o conteúdo do server e manter a forma do arquivo por ferramenta. Sem isso, a Etapa 3 introduz regressão silenciosa.

Melhorias de clareza e precisão

6. “Economia de mais de 80% de tokens de preâmbulo em cada interação” (linha 39)

Impreciso. `CLAUDE.md` / `AGENTS.md` são carregados **uma vez por sessão**, não por interação. “~80% por inicialização de sessão” é honesto; “em cada interação” (linha 20) é incorreto e infla a percepção de ganho.

Ação recomendada: substituir “em cada interação” por “a cada inicialização de sessão” e ajustar a estimativa de 80% para “da pegada de preâmbulo de uma sessão Claude Code/Gemini”.

7. `sys.path` hack em `parametros_projeto.py` (linha 100)

Injetar `sys.path` para suportar a modularização de `scripts/` (`compiladores/` / `validadores/` / `pipeline/`) acopla bootstrap a um arquivo de parâmetros e esconde dependência. Antes de modularizar, convém confirmar que existe **importação cruzada real** entre os scripts — caso contrário a reorganização é estética, sem ganho funcional.

Alternativas mais robustas: `pyproject.toml` com entry points, ou `scripts/__init__.py` + imports absolutos.

8. Plano não cita o `code-review-graph` na transição (Etapa 2)

As regras invioláveis 9 e os hooks (`.claude/settings.json` `PostToolUse/SessionStart`, `.gemini/hooks/crg-update.sh`, `crg-session-start.sh`) dependem da estrutura de diretórios atual. Mover skills para a raiz pode afetar caminhos que os hooks espejam. Além disso, a modularização de `scripts/` (Etapa 3) muda os paths que o `code-review-graph` precisa parsear.

Ação recomendada: adicionar à Etapa 2 uma verificação de que `code-review-graph` continua conseguindo parsear `scripts/*.py` após a modularização, e de que os hooks continuam apontando para o local correto.

9. Aliases `@workspace` / `@skills` / `@brand` / `@output` (linhas 71-76)

São inspiradores, mas **não são suportados por todos os harnesses**. O Claude Code reconhece `@` para importação de arquivo; Gemini CLI e outros não têm semântica definida para `@skills`. Sem verificação ferramenta-por-ferramenta, recomendá-los como convenção pode induzir bugs.

Ação recomendada: constar como “convenção interna de prompts”, não como recurso suportado pelas plataformas.

10. Gates de aceitação subjetivos (Etapa 1, linha 125)

“Validar se os agentes continuam operando normalmente” é impreciso. O projeto já possui `auditar-projeto.py --estrito` e `verificar-consistencia-pipeline.py --estrito` (REGRA 8 — “Scripts são o Árbitro”).

Ação recomendada: declarar esses dois scripts como **gate obrigatório** ao final de cada etapa, em vez de validação subjetiva. Isto alinha o plano à REGRA 8 do próprio `AGENTS.md` e torna cada etapa passável/falhável de forma determinística.

Acertos que valem reiterar

- O diagnóstico de dívida técnica é real: `CLAUDE.md` e `AGENTS.md` são quase idênticos, e isto é desperdício verificável.
 - Especializar cada arquivo de governança por plataforma é a direção correta.
 - Centralizar skills comuns é certo — o **mecanismo** é que precisa revisão (junction > symlink no Windows).
 - O cronograma em 3 etapas é razoável, desde que cada etapa tenha um gate determinístico (item 10).
-

Resumo executivo

#	Item	Severidade	Ação
1	GEMINI.md já tem 46 linhas, não “duplica ~150”	Erro factual	Corrigir diagnóstico (linhas 18-20)
2	4 arquivos de governança omitidos (CODEBUDDY.md , QODER.md , .cursorrules , .windsurfrules)	Omissão	Adicionar à tabela de especialização
3	Etapas 1 pode regressir a tabela de módulos (coluna “Selecionável via /esbocar”, “Textos de Apoio”)	Risco de regressão	Preservar explicitamente o conteúdo
4	Symlink no Windows é frágil, prefira junction	Risco alto	Trocar mecanismo (junction > symlink > cópia)
5	Unificação de .mcp.json ignora schemas diferentes	Risco alto	Documentar schemas ou desistir da unificação
6	“~80% de tokens por interação” — impreciso	Clareza	“Por inicialização de sessão”
7	sys.path hack é frágil	Clareza	pyproject.toml ou scripts/ __init__.py
8	Etapas não citam code-review-graph hooks	Risco oculto	Adicionar verificação de grafo + hooks
9	Aliases @ não são suportados por todos os harnesses	Clareza	“Convenção interna”, não recurso
10	Gates de aceitação subjetivos	Clareza	Usar auditar / verificar-consistencia -- estrito como gate

Próximos passos sugeridos

1. **Imediato:** corrigir o item 1 (afirmação sobre `GEMINI.md`) e o item 2 (omissão dos quatro arquivos de governança) no plano original — são erros factuais que enfraquecem a credibilidade do documento.
2. **Antes da Etapa 2:** validar empiricamente a viabilidade de junctions no ambiente alvo (máquinas de operador + CI) — um quick check de `mklink /J` em um diretório de teste resolve em 5 minutos.
3. **Antes da Etapa 3:** documentar os schemas dos cinco arquivos `.mcp.json` para decidir se a unificação é sequer possível.
4. **Transversal:** adicionar `auditar-projeto.py --estrito` e `verificar-consistencia-pipeline.py --estrito` como gate obrigatório ao final de cada uma das três etapas.

Se desejado, posso (a) **reescrever o plano original** incorporando essas correções (edição cirúrgica no `docs/04-plano-melhoria-estrutural.md`) ou (b) gerar um **diff sugerido** apenas com os trechos a corrigir.

